Salt Creek Masonry Destruction Lab

Nov 4, 2026 · @Sean

Purpose

Build a structural masonry destruction system inside Salt Creek, so the game gets stone walls that crack, hole and collapse believably, and Normandy 1944 starts with a proven set of rules instead of a blank page.

- **For Salt Creek:** dynamite breaches through stone walls, a jail or bank that takes real damage, and stone bridge piers that can fail under the train set piece.
- **For Normandy 1944:** Normandy farmhouses, barns and churches are mostly stone and brick with mortar, and many are half-timbered. Masonry plus Salt Creek's timber work covers most of that village.
- **The rule that makes it transfer:** the structural logic and material numbers live in plain data and a written spec, separate from Godot code (see The portability rule). Godot is the test bench; the rules are the product.

This is a lab, run inside the Salt Creek repo like the existing gunfight, gore and breakage labs. It does not replace or block the farmhouse mission or the explosions coordinator's work; it feeds them.

Materials in scope

Five period-correct masonry materials for an 1882 desert town, each chosen because it maps onto something in Normandy.

Material	Where it appears in Salt Creek	Normandy equivalent
Adobe (sun-dried mud brick)	Older houses, outbuildings, walls	Cob and mud infill in old farm buildings
Rubble stone + lime mortar	Jail, powder magazine, retaining walls, back walls of stores	Farmhouses, barns, field walls
Cut (dressed) stone	Bank front, bridge piers, corners and lintels	Church walls, château, quoins and door surrounds

Material	Where it appears in Salt Creek	Normandy equivalent
Fired brick + lime mortar	Bank front, better commercial buildings, chimneys	Brick houses, chimneys, infill
Timber in masonry	Lintels, floor joists, roof beams resting on stone walls	Half-timbered (colombage) houses, floors, roofs

Out of scope: reinforced concrete. It was barely invented in 1882, so it would be an anachronism in Salt Creek. German bunkers and concrete with rebar are new work for the Unreal phase. The block-and-joint model here still helps, since unreinforced concrete footings and cracks reuse the same crack logic.

Test pieces

Start with one wall in an empty lab scene, and only move to whole buildings once the wall passes. Each piece tests something the last one didn't.

#	Test piece	What it proves	Game use
1	Wall panel, about 4 m × 3 m, one per material	Cracks, holes, chunk sizes, mortar-joint failure	Lab only
2	Wall with a door and a timber lintel	Openings weaken a wall; lintel failure drops the stone above	Every building
3	The jail (rubble stone, one storey, timber roof)	Breach without full collapse; roof sags when a wall section goes	Dynamite jailbreak
4	The bank (cut stone and brick front, rubble back)	Mixed materials in one building; a strong front with weak sides	Bank job, heavy breach
5	Powder magazine (squat stone, dug into a hillside)	A blast from inside; the hillside directs it upward	Mine set piece
6	Stone bridge pier with a timber or iron deck	Load from above; a pier failing drops the span	Train bridge set piece

Each piece is a reusable Normandy reference: piece 3 is a stone barn, piece 4 a village house, piece 6 a Normandy river bridge.

How it should break

Masonry fails at its weakest point, which is almost always the mortar joint, not the stone. That one rule should drive most of the look.

Material	Small hit (bullet, pick, kick)	Big hit (dynamite)	Must never happen
Adobe	Chips and puffs of dust; edges crumble	Blocks break apart and crumble into dirt-like rubble with lots of dust	Clean, sharp-edged chunks
Rubble stone	Mortar flakes; a stone may loosen	Stones separate along mortar; irregular hole with ragged edges	Stones splitting into neat cubes
Cut stone	Chips off faces and corners	Whole blocks shift or fall out along joints; few blocks crack through	A dressed wall shattering like glass
Fired brick	Chips and spalls off the face	Bricks come away in clumps still mortared together, then separate	Every brick flying individually
Timber lintels and beams	Splinters, gouges	Snaps along the grain; the stone above loses support and falls	A beam vanishing or snapping cleanly

Structural behaviour, all materials:

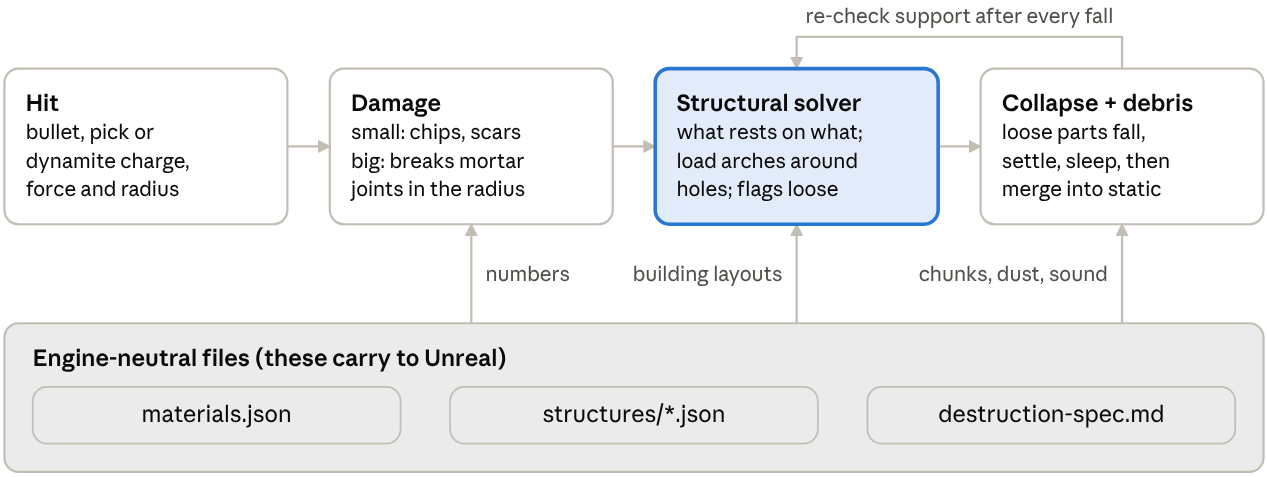
- A hole at the base of a wall makes the stone above it unstable; anything unsupported falls.
- A wall can be badly holed and still stand if load can route around the hole (arching).
- When a wall carrying a roof or floor fails, that roof or floor sags, then drops in the section above it, not the whole building at once.
- Partial collapse is the norm. Full collapse is reserved for a building whose main supports are actually gone.
- Cracks should appear before failure where possible, so the player sees a wall about to go.

Your reference library: photos and video of real masonry and concrete failures from your concrete work are the best target to tune against. A folder in the repo the sessions can look at would help.

System architecture

Four stages run on every hit, and the structural solver in the middle is the part worth the most: it is pure logic, so it ports to Unreal nearly unchanged.

A hit breaks joints; the solver decides what falls



The solver is plain logic with no engine physics; Godot only draws and simulates the falling pieces.

masonry destruction pipeline · 4 stages over engine-neutral data

Each wall is a set of elements (stones, bricks, lintels, beams) joined by mortar joints. A hit breaks joints; the solver checks what is still held up and lets the rest fall; falling pieces trigger another check, which is what makes chain collapses happen.

- **Two scales of damage:** small hits only mark the surface (decals, chipped edges), so 50 bullets never cost what one charge does.
- **Pre-cut pieces:** each element is pre-cut into a few chunk shapes, so breaks look varied without cutting geometry live.
- **Debris lifecycle:** settled rubble goes to sleep, then merges into static geometry, so it can stay all game.

The portability rule

Anything that describes how masonry behaves goes in engine-neutral files; only the code that draws and simulates it is Godot-specific.

Lives outside Godot (carries to Unreal)	Lives in Godot (gets rewritten)
`destruction-spec.md`: the rules in plain English, kept current	GDScript and scenes
`materials.json`: per-material numbers (density, joint strength, chunk size range, dust amount, sound set)	Physics bodies, joints and collision setup
`structures/*.json`: each test building as a list of elements and what rests on what	Meshes cut for Godot, shaders, particles
`benchmarks.md`: each test scene, its numbers and results over time	Performance workarounds specific to Godot
Source assets: meshes, textures, recorded sounds	
Reference library: photos and video of real failures	

Rules for every session:

1. Change a behaviour in the spec and data first, then in code.
2. No magic numbers in GDScript; tuning values are read from `materials.json`.
3. The structural solver (what holds up what) must be plain logic with no Godot physics calls, so it can be ported line for line to C++.
4. Every finished step updates `benchmarks.md`.

Suggested home: a `docs/destruction/` folder in the Salt Creek repo, later copied into the Normandy repo.

Steps

Eight session-sized steps. Each session stops at the end of its step, posts its result and leaves the next step as a "next session" issue, the same way the other labs run.

1. **Spec and data skeleton.** Write `destruction-spec.md`, `materials.json` with first-guess numbers for all five materials, and the JSON format for structures.
   - Done when: Sean has read the spec and approved the material list (a "waiting on Sean" issue).
2. **One rubble-stone wall panel.** Build the wall as stones and mortar joints in an empty lab scene. Hits break joints, not stones.
   - Done when: a dynamite charge at the base blows a ragged hole, and the stone above the hole falls.
3. **Structural solver.** Add the "what holds up what" logic: each element knows what it rests on; unsupported elements fall; load routes around holes.
   - Done when: a hole low in the wall drops what's above it, but a hole with stone arching over it stays up.
4. **All five materials on the wall panel.** Swap the material on the same wall and tune each against the How it should break table.
   - Done when: side-by-side clips of all five, approved by Sean.
5. **Two scales of damage.** Small hits chip and scar the surface without breaking structure; big hits break joints.
   - Done when: 50 rifle hits pockmark a wall without collapsing it; one dynamite charge breaches it.
6. **Debris lifecycle.** Rubble settles, goes to sleep, then merges into static geometry so it stays without costing frame rate.
   - Done when: rubble from three breaches is still there after a full in-game day, with no frame rate drop.
7. **Dust and sound.** Dust amount and sound set come from the material data; a collapsing adobe wall and a stone wall sound different. Builds on the explosions coordinator's dynamite audio.
   - Done when: Sean can tell the material by ear with eyes closed.
8. **The jail.** First full building: rubble stone with a timber roof. Breach a wall, roof sags in that section, the rest stands.
   - Done when: it passes the jail benchmark below and Sean has played the breach.

After step 8, the bank, powder magazine, bridge pier and (proposed) Baron's house each become their own step, chosen by which Salt Creek mission needs them first.

Benchmarks

All tests run on the Shadow PC (RTX 2000 Ada 16 GB, 8 vCores, 16 GB RAM), which is close to a modest player machine. The frame rate targets below are proposed starting points for Sean to confirm.

Test scene	What happens	Pass target
Wall panel breach	One dynamite charge at the base of a 4 m × 3 m rubble-stone wall	Never below 45 fps during the blast; back to normal within 3 s
Wall stress	50 rifle hits, then 3 charges in a row	No crash; debris count stays under the cap set in step 6
Jail breach	Breach one jail wall; roof section sags and drops	Never below 40 fps; the rest of the jail stands
Persistence	Three breaches, then a full in-game day passes	Rubble still there; frame rate same as before the breaches
Street test	Jail breach on the main street with NPCs and town clutter loaded	Never below 30 fps

When a new PC arrives, re-run the same scenes on it and log both results side by side in `benchmarks.md`. That table is what gets compared against the same scenes in Unreal later.

Decisions

Decided

- Run the lab inside the Salt Creek repo, in Godot, alongside the existing labs.
- Period-correct masonry only: adobe, rubble stone, cut stone, fired brick, timber in masonry. No reinforced concrete.
- Rules and numbers live outside Godot code so they carry to Normandy 1944.
- **Finale:** after the train is destroyed, the player raids the Baron's barricaded masonry house, blows open a wall and arrests him. This makes the Baron's house the lab's final test piece and the toughest breach in the game.

Proposed (Sean to confirm)

- The Baron′s house is fired brick or cut stone (Victorian/Italianate), or a thick-walled stone or adobe hacienda. Barricades inside (shutters, furniture, timber bracing) use the existing breakage systems.
- Rubble stone first, because it covers the jail, the powder magazine and Normandy farmhouses.
- The frame rate targets in Benchmarks.
- A separate destruction-lab session, or folding it into the explosions coordinator′s scope.

Open

- Which Salt Creek buildings are actually masonry, beyond the jail and bank.
- Brick or hacienda for the Baron′s house.
- Whether fire after blasts (parked) interacts with timber in masonry buildings later.
- Where the reference library of real failure photos and video lives.